

SISTEMI OPERATIVI con LABORATORIO

Progetto della parte di Laboratorio – a.a. 2025-26

Progetto “Esecuzione Remota”

Realizzare in linguaggio C ed in ambiente *Unix/Linux* un'applicazione denominata “**Esecuzione Remota**” che consenta l'esecuzione remota di comandi tramite architettura **client-server**. L'applicazione è composta dai seguenti processi:

- processo **server**
- più processi **esecutori**
- più processi **client**

=====

Descrizione dell'applicazione

Tutti i processi componenti l'applicazione sono a sé stanti, cioè NON sono *thread/processi leggeri* di un altro di essi. L'applicazione è composta da due file eseguibili diversi, uno per il processo **server** ed uno per i processi **client**. **Server** e **client** sono avviati da *shell*; gli **esecutori** sono creati (*fork(2)*) dal **server** come descritto più avanti.

In estrema sintesi:

- l'applicazione si avvia eseguendo il processo **server**, che rimane in continuazione in attesa di nuove connessioni dai **client**
- in altri terminali l'utente avvia più processi **client**, ognuno dei quali apre una connessione col **server** tramite uno tra i seguenti strumenti di IPC visti a lezione: *sockets*, *pipes* o *shared memory*
- per ogni client connesso il **server** avvia con *fork(2)* un nuovo processo **esecutore** che gestirà lo scambio di dati col **client**
- ogni **client** invia comandi *shell*, richiesti dall'utente, al corrispondente **esecutore** che li esegue
- ogni **esecutore** ritorna l'output dei comandi eseguiti al corrispondente **client** che lo visualizza all'utente

Si ribadisce che, per la comunicazione tra **server** e **client**, e tra **client** ed **esecutore**, si deve utilizzare esclusivamente una delle forme di *IPC* citate sopra. NON utilizzare scambio di dati attraverso file (*ordinari*) su disco (ovviamente sono invece ammessi *sockets* locali o *named pipes*).

Di seguito la descrizione di ogni processo.

Processo server:

Il processo **server** esegue in ciclo le seguenti operazioni:

1. attende richieste di apertura di connessioni dai vari **client**
2. per ogni **client** connesso, il **server** crea un processo **esecutore** tramite il richiamo di *fork(2)* ed assegna tale **esecutore** al **client** che ha richiesto la connessione
3. da questo punto in poi il server non dialoga più col **client**. Ogni **client** dialoga, sempre con uno degli strumenti di IPC citati sopra, solo con l'**esecutore** assegnatogli e viceversa
4. si rimette in ascolto di altre richieste di connessione (punto 1.)

Processo client:

Ogni **client** apre una connessione col **server**. Dopodiché esegue in ciclo le seguenti operazioni:

1. richiede all'utente di inserire da terminale un comando da eseguire in remoto
2. lo invia (con uno degli strumenti di IPC citati sopra) al corrispondente **esecutore**
3. attende che l'**esecutore** gli ritorni l'output del comando
4. visualizza all'utente l'output ritornato
5. richiede all'utente di inserire un nuovo comando (punto 1.)

Nota: se l'utente, al punto 1., inserisce la stringa "exit" come comando, il **client** invia tale stringa all'**esecutore** e termina

Processo esecutore:

ogni **esecutore** esegue in ciclo le seguenti operazioni:

1. attende che il **client** assegnatogli invii un comando da eseguire
2. esegue il comando ricevuto. Tale esecuzione **DEVE** essere attuata con *fork(2)* ed una delle funzioni elencate in *man 3 exec* invocate in sequenza. **NON**, per esempio, con *system(3)*
3. invia l'output del comando eseguito al **client** (con uno degli strumenti di IPC citati sopra)
4. attende un nuovo comando (punto 1.)

Nota: se il **client** invia al punto 1. la stringa "exit" come comando, l'**esecutore** termina

N.B.: non sono richieste:

- l'esecuzione di comandi CON argomenti
- l'esecuzione di comandi che rimangano in attesa dello *standard input* (es.: *cat* senza argomenti)
- l'esecuzione di comandi interni della *shell*

Sarà comunque considerata nota di merito l'implementazione dell'esecuzione di comandi con argomenti (a tal scopo può esservi utile *strtok(3)*).

=====

Terminazione dell'applicazione:

Tutti i processi componenti l'applicazione devono terminare invocando *return()* o *exit()*, e non essere terminati "bruscamente" dal sistema (per esempio perché ricevono un *signal* non gestito).

- Terminazione del **server**
 - Il **server** termina solo se riceve *SIGTERM*
 - prima di terminare **DEVE** informare eventuali processi **esecutori** ancora attivi che l'applicazione si sta chiudendo e che quindi essi stessi devono terminare. Lo studente può attuare tale comunicazione tra **server** ed **esecutori** con qualsiasi strumento (suggerimento: si ricorda che, per ottenere il *PGID* (*Process Group ID*) si può utilizzare *getpgrp(2)*. Ma potete anche scegliere di non utilizzarla...).
 - Il **server** **DEVE** attendere la terminazione di tutti i processi **esecutori** prima di terminare lui stesso
- Terminazione degli **esecutori**:
 - un **esecutore** termina se:
 - riceve la richiesta di terminazione dal **server**
 - riceve la stringa "exit" dal client come comando da eseguire
- Terminazione dei **client**:
 - Il **client** termina se:
 - l'utente inserisce la stringa "exit"
 - rileva che l'**esecutore** corrispondente ha chiuso la comunicazione, per esempio all'atto di inviare un comando da eseguire
 - se l'utente inserisce la stringa "exit" il **client** la invia **all'esecutore** prima di terminare

Ulteriori specifiche:

- **client**, **server** ed **esecutore** devono ignorare *SIGINT* e *SIGQUIT*
- lo studente dovrà fornire anche uno [*script Bash*](#) che compila e avvia l'applicazione
- oltre ai processi descritti nell'applicazione lo studente può utilizzare **ulteriori** processi o *threads* che ritenga funzionali per l'applicazione, **sempre nel rispetto delle presenti specifiche**
- è gradito l'inserimento di commenti nel codice che ne facilitino la comprensione
- il progetto per funzionare non deve richiedere l'esecuzione dall'utente *root*, né richiedere di essere eseguito in ambiente grafico
- evitare attese attive (es.: cicli che impegnino continuamente la *CPU*)
- evitare attese passive non necessarie

NOTE FINALI:

- esula dagli scopi di questo laboratorio trattare aspetti inerenti la sicurezza. Eseguire comandi in remoto senza alcun controllo comporta problemi di sicurezza, non ce ne curiamo in questa sede
- si noti che, a seconda dello strumento di *IPC* utilizzato, **server** e **client** possono stare anche in macchine diverse e dialogare attraverso rete

INDICAZIONI PER LA CONSEGNA:

Il progetto deve essere spedito via e-mail al docente all'indirizzo fabio.rossi@unipg.it, oppure messo a disposizione su un qualsiasi *cloud* e reso scaricabile tramite link da inviare sempre al suddetto indirizzo.

I progetti redatti secondo queste specifiche possono essere consegnati:

- dal 1 Giugno 2026
- al 31 Maggio 2027

La consegna deve avvenire tassativamente almeno una settimana prima della data dell'esame orale. Il formato può essere tar, tgz o zip.